**ChatGPT said:**

## Tutoriel Vue.js 3 – Les fondamentaux avec CDN

Débutant

Sans installation

Dans ce tutoriel, vous allez découvrir les principales fonctionnalités de Vue.js 3 en utilisant uniquement un fichier HTML et un CDN.

L'objectif est de comprendre comment Vue.js permet de créer des interfaces web interactives, sans avoir besoin de manipuler directement le DOM.

À la fin de ce tutoriel, vous saurez utiliser :

1. `createApp()` – créer une application Vue.
2. `data()` – définir les données de l'application.
3. `methods` – créer des fonctions.
4. `computed` – calculer des valeurs automatiquement.
5. `v-if` – afficher ou masquer des éléments.
6. `v-for` – parcourir une liste.
7. `v-model` – synchroniser les données et les champs de formulaire.
8. La gestion des événements – réagir aux actions de l'utilisateur.

### 0. Préparation : votre première page Vue.js

Créez un fichier `index.html` et copiez le code suivant :

HTML

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Apprendre Vue.js</title>
</head>
<body>

    <div id="app">
        <h1>{{ message }}</h1>
    </div>

    <!-- Chargement de Vue.js 3 depuis le CDN -->
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>

    <script>
        const { createApp } = Vue

        createApp({
            data() {
                return {
                    message: 'Bonjour Vue.js !'
                }
            }
        }).mount('#app')
    </script>

</body>
</html>
```

Ouvrez le fichier dans votre navigateur.

Vous devriez voir :

**Bonjour Vue.js !**

Votre première application Vue.js

Vous venez de créer votre première application Vue.js !

### 1. `createApp()` – Créer une application Vue

`createApp()` est la fonction qui permet de créer une nouvelle application Vue.js.

Exemple :

JavaScript

```
const { createApp } = Vue

createApp({
    // Configuration de l'application
}).mount('#app')
```

Explications :

- `Vue` est l'objet global fourni par le CDN.
- `createApp()` crée une instance d'application Vue.
- L'objet passé à `createApp()` contient la configuration de votre application.
- `.mount('#app')` indique à Vue où l'application doit être activée dans le HTML.

Dans cet exemple :

HTML

```
<div id="app">
    <h1>Bonjour !</h1>
</div>
```

Vue prend le contrôle du contenu de l'élément `#app`.

**À retenir :** `createApp()` crée l'application et `mount()` l'attache à la page HTML.

### 2. `data()` – Définir les données

La fonction `data()` permet de définir les données réactives de votre application.

Une donnée réactive est une donnée dont les modifications sont automatiquement répercutées dans l'interface.

Exemple :

JavaScript

```
createApp({
    data() {
        return {
            product: 'Chaussettes',
            price: 12,
            quantity: 2
        }
    }
}).mount('#app')
```

Dans le HTML :

HTML

```
<div id="app">
    <h1>{{ product }}</h1>
    <p>Prix : {{ price }} €</p>
```

```
    <p>Quantité : {{ quantity }}</p>
</div>
```

Résultat :

**Chaussettes**

Prix : 12 €

Quantité : 2

Les expressions entre doubles accolades `{{ }}` permettent d'afficher les valeurs des données Vue dans le HTML.

**Pourquoi parle-t-on de réactivité ?**

Si la quantité passe de 2 à 3, Vue met automatiquement à jour l'affichage correspondant.

Vous n'avez pas besoin de modifier manuellement le HTML.

### 3. `methods` – Définir des fonctions

La propriété `methods` permet de définir des fonctions utilisables dans votre application.

Par exemple, vous pouvez créer une fonction qui augmente la quantité d'un produit.

JavaScript

```
createApp({
    data() {
        return {
            quantity: 1
        }
    },

    methods: {
        increaseQuantity() {
            this.quantity++
        },

        decreaseQuantity() {
            this.quantity--
        }
    }
}).mount('#app')
```

Dans le HTML :

HTML

```
<div id="app">
    <p>Quantité : {{ quantity }}</p>

    <button @click="increaseQuantity">
        +
    </button>

    <button @click="decreaseQuantity">
        -
    </button>
</div>
```

Résultat interactif :

**Quantité du produit**

- 1 +

Démonstration du comportement attendu dans Vue.js.

Quelques explications importantes :

- `methods` contient les fonctions de votre application.
- `increaseQuantity()` augmente la quantité.
- `decreaseQuantity()` diminue la quantité.
- `this.quantity` permet d'accéder à la propriété `quantity` définie dans `data()`.

**À retenir :** `this` fait référence à l'instance du composant Vue dans cette syntaxe Options API.

### 4. `computed` – Calculer des valeurs automatiquement

La propriété `computed` permet de définir des valeurs calculées à partir d'autres données réactives.

Imaginons que vous souhaitiez calculer automatiquement le montant total d'un produit.

La formule est :

Prix total = Prix unitaire × Quantité

Exemple :

JavaScript

```
createApp({
    data() {
        return {
            price: 12,
            quantity: 2
        }
    },

    computed: {
        totalPrice() {
            return this.price * this.quantity
        }
    }
}).mount('#app')
```

Dans le HTML :

HTML

```
<div id="app">
    <p>Prix unitaire : {{ price }} €</p>
    <p>Quantité : {{ quantity }}</p>

    <h3>Total : {{ totalPrice }} €</h3>
</div>
```

Résultat :

**Calculateur de prix**

Prix unitaire **12 €**

Quantité **2**

**Total** **24 €**

Exemple interactif du calcul réactif.

Lorsque `price` ou `quantity` change, Vue recalcule automatiquement `totalPrice`.

**Quelle différence entre `methods` et `computed` ?**

| `methods` | `computed` |
|---|---|
| Exécute une fonction | Calcule une valeur |
| Appelé explicitement | Utilisé comme une propriété |
| Pas de cache automatique | Résultat mis en cache |
| Idéal pour les actions | Idéal pour les calculs |

Un `computed` est recalculé lorsque l'une de ses dépendances réactives change.

### 5. `v-if` – Afficher ou masquer des éléments

`v-if` est une directive Vue qui permet d'afficher un élément HTML uniquement si une condition est vraie.

Exemple :

JavaScript

```
createApp({
    data() {
        return {
            isAvailable: true
        }
    }
}).mount('#app')
```

Dans le HTML :

HTML

```
<div id="app">
    <p v-if="isAvailable">
        Produit disponible !
    </p>

    <p v-else>
        Produit indisponible.
    </p>
</div>
```

Résultat lorsque `isAvailable` vaut `true` :

**Produit disponible !**

Résultat lorsque `isAvailable` vaut `false` :

**Produit indisponible.**

Vous pouvez également utiliser `v-else-if` pour tester plusieurs conditions.

Exemple :

HTML

```
<p v-if="stock > 10">
    Stock suffisant
</p>

<p v-else-if="stock > 0">
    Stock limité
```

```
</p>

<p v-else>
    Rupture de stock
</p>
```

**À retenir :** `v-if` ajoute ou retire réellement l'élément du DOM selon la condition.

Il existe également `v-show`, qui masque un élément avec CSS ( `display: none` ) sans le supprimer du DOM.

### 6. `v-for` – Parcourir une liste

`v-for` permet de parcourir une collection de données et de générer automatiquement plusieurs éléments HTML.

Imaginons que vous ayez une liste de produits.

Dans JavaScript :

JavaScript

```
createApp({
    data() {
        return {
            products: [
                'Chaussettes',
                'Chaussures',
                'T-shirt',
                'Pull'
            ]
        }
    }
}).mount('#app')
```

Dans le HTML :

HTML

```
<div id="app">
    <h2>Nos produits</h2>

    <ul>
        <li v-for="product in products">
            {{ product }}
        </li>
    </ul>
</div>
```

Résultat :

**Nos produits**

- Chaussettes
- Chaussures
- T-shirt
- Pull

Explications :

- `products` est le tableau défini dans `data()` .
- `product` représente l'élément courant du tableau.
- `in` indique que Vue doit parcourir la collection.
- Vue génère un élément `<li>` pour chaque produit.

**Exemple avec des objets**

Vous pouvez également parcourir un tableau d'objets :

JavaScript

```
data() {
    return {
        products: [
            { id: 1, name: 'Chaussettes', price: 12 },
            { id: 2, name: 'Chaussures', price: 80 },
            { id: 3, name: 'T-shirt', price: 25 }
        ]
    }
}
```

Dans le HTML :

HTML

```
<div id="app">
    <ul>
        <li v-for="product in products"
            :key="product.id">

            {{ product.name }} -
            {{ product.price }} €

        </li>
    </ul>
</div>
```

Remarquez `:key="product.id"`.

Il s'agit d'un attribut important qui permet à Vue d'identifier chaque élément de la liste et de mettre à jour efficacement le DOM.

**À retenir :** `v-for` est particulièrement utile pour afficher des produits, des utilisateurs, des articles ou toute autre liste dynamique.

### 7. `v-model` – Synchroniser les données et les formulaires

`v-model` permet de créer une liaison entre une donnée Vue et un champ de formulaire HTML.

C'est ce que l'on appelle une liaison bidirectionnelle (*two-way data binding*).

Exemple :

JavaScript

```
createApp({
    data() {
        return {
            username: ''
        }
    }
}).mount('#app')
```

Dans le HTML :

HTML

```
<div id="app">
    <label>Votre nom :</label>

    <input type="text" v-model="username">

    <p>Bonjour {{ username }} !</p>
</div>
```

Que se passe-t-il ?

1. L'utilisateur saisit son nom dans le champ.
2. Vue met automatiquement à jour `username`.
3. Le texte affiché dans le paragraphe est automatiquement actualisé.

**Essayez vous-même**

**Formulaire de démonstration**

Votre nom :

Saisissez votre nom...

Saisissez votre nom pour voir le résultat.

Démonstration du fonctionnement de v-model.

Vous pouvez également utiliser `v-model` avec d'autres champs :

HTML

```
<!-- Champ texte -->
<input v-model="username">

<!-- Case à cocher -->
<input type="checkbox" v-model="accepted">

<!-- Liste déroulante -->
<select v-model="selectedCountry">
    <option value="fr">France</option>
    <option value="ch">Suisse</option>
    <option value="be">Belgique</option>
</select>

<!-- Zone de texte -->
<textarea v-model="description"></textarea>
```

**À retenir :** `v-model` évite de devoir récupérer manuellement la valeur d'un champ HTML et de mettre à jour les données JavaScript.

### 8. Gestion des événements – Réagir aux actions de l'utilisateur

Une application web doit pouvoir réagir aux actions de ses utilisateurs :

- Clic sur un bouton.
- Saisie au clavier.
- Sélection d'une option.
- Soumission d'un formulaire.

Vue utilise la directive `v-on` pour gérer les événements.

Il existe également une syntaxe raccourcie : `@`.

Par exemple :

HTML

```
<button v-on:click="sayHello">
    Cliquez ici
</button>
```

Est équivalent à :

HTML

```
<button @click="sayHello">
    Cliquez ici
</button>
```

**Exemple : réagir à un clic**

JavaScript :

JavaScript

```
createApp({
    data() {
        return {
            message: 'Bonjour !'
        }
    },

    methods: {
        sayHello() {
            alert('Bonjour et bienvenue !')
        }
    }
}).mount('#app')
```

HTML :

HTML

```
<div id="app">
    <p>{{ message }}</p>

    <button @click="sayHello">
        Cliquez ici
    </button>
</div>
```

Lorsque l'utilisateur clique sur le bouton, Vue exécute la méthode `sayHello()`.

**Quelques événements courants**

| Syntaxe | Événement |
|---|---|
| `@click` | Clic sur un élément |
| `@input` | Modification d'un champ |
| `@change` | Changement de valeur |
| `@submit` | Soumission d'un formulaire |
| `@keyup` | Relâchement d'une touche |
| `@mouseover` | Passage de la souris |

**Exemple : compteur de clics**

JavaScript

```
createApp({
    data() {
        return {
            clicks: 0
        }
    },

    methods: {
        increment() {
            this.clicks++
```

```
        }
    }
}).mount('#app')
```

HTML :

HTML

```
<div id="app">
    <p>Nombre de clics : {{ clicks }}</p>

    <button @click="increment">
        Cliquez-moi !
    </button>
</div>
```




Vous pouvez également transmettre des paramètres à une méthode :

HTML

```
<button @click="addProduct('Chaussettes')">
    Ajouter les chaussettes
</button>
```

JavaScript

```
methods: {
    addProduct(name) {
        alert('Produit ajouté : ' + name)
    }
}
```

**À retenir :** `@click` est simplement une manière concise d'associer une action utilisateur à une méthode JavaScript.

---

### 9. Exemple pratique : créer une petite boutique

Maintenant que vous connaissez les principales fonctionnalités, voici un exemple qui les combine dans une seule application.

Vous allez créer une petite page présentant des produits avec :

- Une liste de produits.
- Un prix et un stock.
- Un champ de recherche.
- Un bouton pour ajouter des produits au panier.
- Un total calculé automatiquement.
- Un affichage conditionnel selon la disponibilité.

HTML

```
<!DOCTYPE html>
<html lang="fr">
```

```
<head>
    <meta charset="UTF-8">
    <title>Ma boutique Vue.js</title>
</head>
<body>

<div id="app">

    <h1>{{ title }}</h1>

    <!-- Recherche : v-model -->
    <input
        type="text"
        v-model="search"
        placeholder="Rechercher un produit..."
    >

    <!-- Liste des produits : v-for -->
    <h2>Nos produits</h2>

    <div v-for="product in filteredProducts"
         :key="product.id">

        <h3>{{ product.name }}</h3>

        <p>Prix : {{ product.price }} €</p>
```

**Ce que vous venez d'utiliser**

| Fonctionnalité | Utilisation dans la boutique |
|---|---|
| `createApp()` | Création de l'application |
| `data()` | Produits, panier et recherche |
| `methods` | Ajout au panier et réinitialisation |
| `computed` | Calcul du total et filtrage |
| `v-if` / `v-else` | Disponibilité des produits |
| `v-for` | Affichage de la liste des produits |
| `v-model` | Champ de recherche |
| `@click` | Boutons d'action |

*Remarque : cet exemple est pédagogique. Il utilise des références aux objets produits dans le panier et ne gère pas encore les quantités regroupées ni la persistance des données.*

---

## 10. Petit exercice pour vérifier vos connaissances

Essayez de répondre aux cinq questions suivantes.

**Quiz Vue.js** 0/5 réponses

**1. Quelle fonction permet de créer une application Vue ?**

- ○ data()
- ○ createApp()
- ○ methods

**2. Où déclare-t-on les données réactives dans l'Options API ?**

- ○ Dans data()
- ○ Dans computed
- ○ Dans mount()

**3. Quelle directive permet de parcourir un tableau ?**

- v-if
- v-model
- v-for

**4. Quelle directive synchronise un champ de formulaire avec une donnée ?**

- v-on
- v-model
- v-show

**5. Quelle syntaxe permet de réagir à un clic ?**

- @click
- v-for
- {{ click }}

Corriger mes réponses

### 11. Pour aller plus loin

Une fois ces notions maîtrisées, les prochaines étapes naturelles sont :

1. **Les composants Vue.js** : découper votre application en plusieurs éléments réutilisables.
2. **Les props** : transmettre des données d'un composant parent à un composant enfant.
3. **Les événements personnalisés** : communiquer d'un composant enfant vers son parent.
4. **L'API REST et** `fetch()` : récupérer des données depuis un serveur.
5. **Vue Router** : gérer plusieurs pages.
6. **Pinia** : partager des données entre plusieurs composants.

Documentation officielle : Guide Vue.js 3 ↗.

**Mon conseil :** prenez le temps de modifier les exemples de ce tutoriel. Changez les valeurs, ajoutez des propriétés, supprimez des méthodes et observez les résultats. C'est une excellente façon de comprendre réellement le fonctionnement de Vue.js.